

Assignment 3: Experimental Engineering (Performance / Mutation / Chaos Testing)

Deadline: Week 7

Objective

This assignment performs **experimental testing** to evaluate system robustness, performance, and fault tolerance. By completing this assignment, you will:

- Assess **system reliability, resilience, and performance** under various controlled conditions.
- Generate **quantitative metrics** for your final research paper.
- Gain hands-on experience with **advanced testing techniques** beyond functional and automation testing.
- Develop the ability to **analyze system behavior under stress and unexpected conditions**, a critical skill in modern QA and DevOps practices.

Assignment Tasks

1. Performance Testing

Evaluate the system's performance under **expected and extreme load conditions** to identify bottlenecks, measure resource usage, and ensure the system meets performance requirements.

Tasks and Subtasks

Task	Subtasks / Steps	Tools / Techniques	Expected Metrics / Outputs
Design Load & Stress Tests	- Identify 3 high-risk modules/components based on midterm risk analysis - Define test scenarios: normal load, peak load, spike load, endurance test - Determine number of concurrent users/requests and test duration	JMeter, K6, Postman Newman	- Test plan with scenarios, expected thresholds, and execution parameters

Execute Tests	- Run load tests simulating normal user traffic - Gradually increase load to peak and stress levels - Record real-time system behavior and logs	JMeter, K6, Postman Newman, CI/CD pipeline	- Raw performance logs - Execution screenshots - System response under load
Collect Metrics	- Measure response times (average, median, 95th percentile) - Record throughput (requests per second) - Track error rates (HTTP errors, failed operations) - Monitor resource usage (CPU, memory, I/O)	Monitoring tools: Grafana, Prometheus, Android Profiler (mobile), system monitors	- Response time table: Average / Median / 95th percentile - Throughput over time - Error rate % - CPU/Memory/IO usage charts
Analyze & Identify Bottlenecks	- Compare observed performance with expected thresholds - Identify modules causing slow response or high resource consumption - Suggest optimization strategies (e.g., caching, query optimization, code refactoring)	Performance analysis tools, logs, profiling tools	- Bottleneck report with identified modules, impact, and optimization recommendations

Documentation Requirements

- **Test Plan:** Include scenarios, load levels, duration, and expected results.
- **Execution Logs:** Provide screenshots, raw logs, and any CI/CD pipeline outputs.
- **Metrics Visualization:** Use tables and graphs for response times, throughput, and errors.
- **Analysis & Recommendations:** Summarize bottlenecks, root causes, and suggested improvements.

2. Mutation Testing

Goal:

Assess the **effectiveness of your automated test suite** by introducing controlled code faults (mutants) and measuring whether the test suite detects them. This helps identify weaknesses in test coverage and improves overall QA quality.

Tasks and Subtasks

Task	Subtasks / Steps	Tools / Techniques	Expected Metrics / Outputs
Introduce Controlled Mutations	- Identify critical modules/components from the midterm project. - Create mutants by introducing small, deliberate changes, e.g.: • Changing logical operators (e.g., <code>==</code> → <code>!=</code>) • Altering constants (e.g., <code>MAX_RETRIES = 3</code> → <code>MAX_RETRIES = 0</code>) • Removing or commenting out function calls • Modifying return values - Ensure mutations are realistic and meaningful	Mutation testing frameworks: PIT (Java), MutPy (Python), Stryker (JavaScript)	- List of all mutants created with description of changes
Execute Automated Tests Against Mutants	- Run the existing automated test suite for each mutant - Capture test results: which mutants cause test failures (killed) and which pass undetected (survived) - Log any unexpected behavior or errors	Existing CI/CD pipeline with automated tests, Mutation testing tools	- Detailed log of mutant executions - Status for each mutant: Killed or Survived
Calculate Mutation Score			

- Summarize the overall effectiveness of the test suite	Manual calculation or built-in mutation tool reports	- Mutation Score (%) for the tested modules	
Analyze Results & Identify Gaps	- Identify surviving mutants and analyze why they were not detected - Highlight gaps in test coverage or missing test cases - Suggest improvements to the test suite (new test cases, assertions, input scenarios)	Mutation reports, test coverage tools (JaCoCo, Coverage.py)	- Analysis table showing surviving mutants, their impact, and recommended improvements - Suggested additions to the test suite

Example Mutation Metrics Table

Module / Component	Mutant Type	Mutants Created	Mutants Killed	Mutants Survived	Mutation Score (%)
User Authentication	Logical operator change	5	4	1	80%
Payment Processing	Constant alteration	3	2	1	66.7%
Data Validation	Function removal	4	3	1	75%
API Request Handler	Return value modification	6	5	1	83.3%
Overall	—	18	14	4	77.8%

Documentation Requirements

1. **Mutation Plan:** Describe which modules were mutated, types of mutations, and rationale.
2. **Test Execution Logs:** Include detailed output from the test suite against mutants.
3. **Mutation Score Calculation:** Show calculations clearly for each module and overall system.
4. **Analysis & Recommendations:** Highlight surviving mutants and provide actionable steps to improve test coverage.

 **Tip:** Start with a **small number of mutants per module** to validate your mutation setup before scaling up. This prevents overwhelming the system and ensures manageable analysis.

3. Chaos / Fault Injection Testing

Goal:

Evaluate the **system's robustness, resilience, and fault tolerance** by simulating failures in critical modules and observing how the system handles, recovers, and mitigates the impact of faults.

Tasks and Subtasks

Task	Subtasks / Steps	Tools / Techniques	Expected Metrics / Outputs
Simulate Failures in High-Risk Modules	- Identify high-risk modules/components based on midterm risk analysis - Introduce controlled faults, including: • API downtime: temporarily shut down critical services • Database failure: simulate unavailability or slow responses • Network latency/packet loss: throttle network speed, inject delays • Resource exhaustion: max out CPU, memory, or I/O	Chaos Mesh, Gremlin, Simian Army, Postman, Custom scripts, Network Link Conditioner	- List of all injected faults with type, duration, and affected modules
Observe System Behavior	- Monitor error handling and logging mechanisms - Check recovery mechanisms: retries, failovers, graceful degradation - Test impact on critical system functions (login, payment, data processing)	Monitoring tools (Prometheus, Grafana, CloudWatch), logs, automated scripts	- Behavior report for each failure scenario - Screenshots and logs showing system response and recovery

Record Metrics	- System availability (%) during fault injection - Mean recovery time (MTTR) for each failure type - Error/failure propagation across dependent modules - Any service degradation or data inconsistencies	Grafana, Prometheus, CI/CD pipeline dashboards, custom scripts	- Metrics table for each failure scenario with availability, recovery time, and error propagation
Document Lessons Learned	- Analyze which modules handled faults well and which failed - Identify gaps in fault tolerance or error handling - Provide recommendations to improve resilience and robustness	Observations, metrics analysis, documentation	- Summary of lessons learned with actionable recommendations for QA and development teams

Documentation Requirements

1. **Chaos Testing Plan:** List modules, failure types, and duration of simulated faults.
2. **Execution Logs:** Include monitoring dashboards, system logs, and screenshots showing fault injection and system behavior.
3. **Metrics Summary:** Provide tables and graphs for availability, recovery time, and error propagation.
4. **Analysis & Lessons Learned:** Describe strengths, weaknesses, and suggested improvements in resilience strategies.

 **Tip:** Start with **short, isolated fault injections** before attempting long or combined scenarios to avoid catastrophic failures. Record everything precisely for reproducibility in your final research paper.

4. Documentation & Analysis

Goal:

Ensure that all experimental testing is **thoroughly recorded, analyzed, and visualized** so that results are reproducible, interpretable, and ready for inclusion in the final research paper.

Tasks and Subtasks

Task	Subtasks / Steps	Tools / Techniques	Expected Outputs / Deliverables
Record Experimental Setup Details	- Document the environment: OS, hardware, network conditions, and system configuration - Record tool versions, frameworks, and dependencies - Include all scripts and configuration files used for performance, mutation, and chaos testing - Capture CI/CD pipeline setup for test automation	Markdown / LaTeX documentation, Git repository, screenshots of pipeline	- Complete setup documentation with tool versions, scripts, and configurations - Reproducible experimental setup
Capture Metrics & Visualize Results	- Collect quantitative metrics from all tests: • Performance: response time, throughput, CPU/memory usage, error rates • Mutation: mutation score, surviving mutants • Chaos: system availability, recovery time, error propagation - Visualize metrics using: • Tables • Graphs (line, bar, scatter) • Screenshots of dashboards and logs	Excel, Google Sheets, Grafana, Prometheus, Matplotlib/Seaborn	- Tables and graphs summarizing experimental results - Screenshots from monitoring tools and CI/CD pipelines

Compare Observed vs Expected Behavior	- Review risk-based analysis from midterm project - Compare actual experimental outcomes with expected thresholds and risk predictions - Identify discrepancies and explain root causes	Analytical tools, Excel, Documentation tools	- Comparative analysis tables and narrative explaining deviations - Highlighted risks that were underestimated or overestimated
Include Recommendations	- Suggest improvements to QA processes and automation scripts - Recommend additional test scenarios or modules for future experiments - Identify areas to improve system robustness and resilience	Documentation tools	- Recommendations section in report with actionable steps for QA and development teams

Deliverables

Deliverable	Content / Requirements
Experimental Testing Report (4–6 pages)	- Description of the system and tested modules (high-risk from midterm) - Performance testing methodology, execution, and results - Mutation testing methodology, mutants created, and mutation score - Chaos/fault injection testing methodology, execution, and observations - Comparative analysis of expected vs actual system behavior - Lessons learned and recommendations
Updated Automation Repository / Scripts	- Include all additional scripts for performance, mutation, and chaos testing - Ensure integration with CI/CD pipelines for reproducibility

Metrics Summary	- Quantitative metrics: • Coverage % • Response times • Mutation scores • Error/failure rates • Recovery times - Visualizations: graphs, tables, and screenshots
------------------------	---